

Data Structures



Lecture 4 Linked List Basics + Functions

Prantik Paul [PNP]

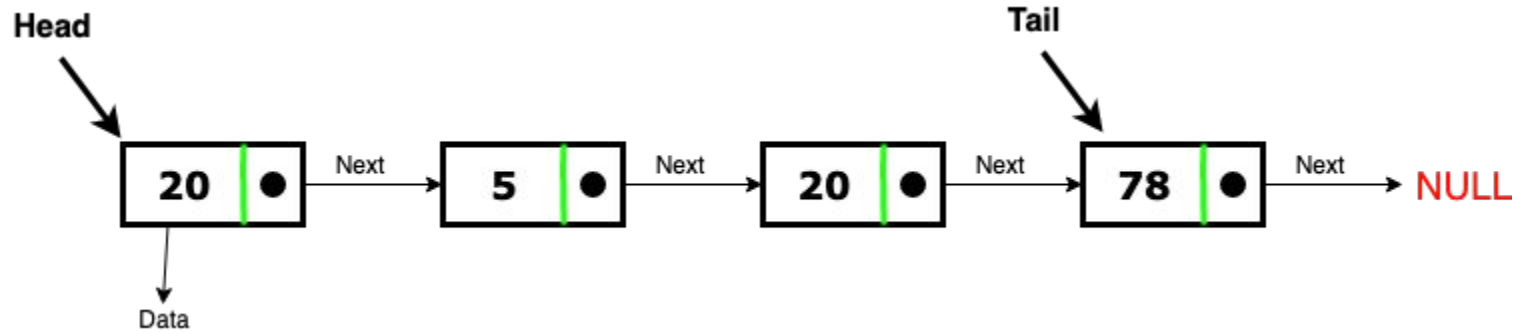
Lecturer

Department of Computer Science and Engineering
BRAC University

Array Problems

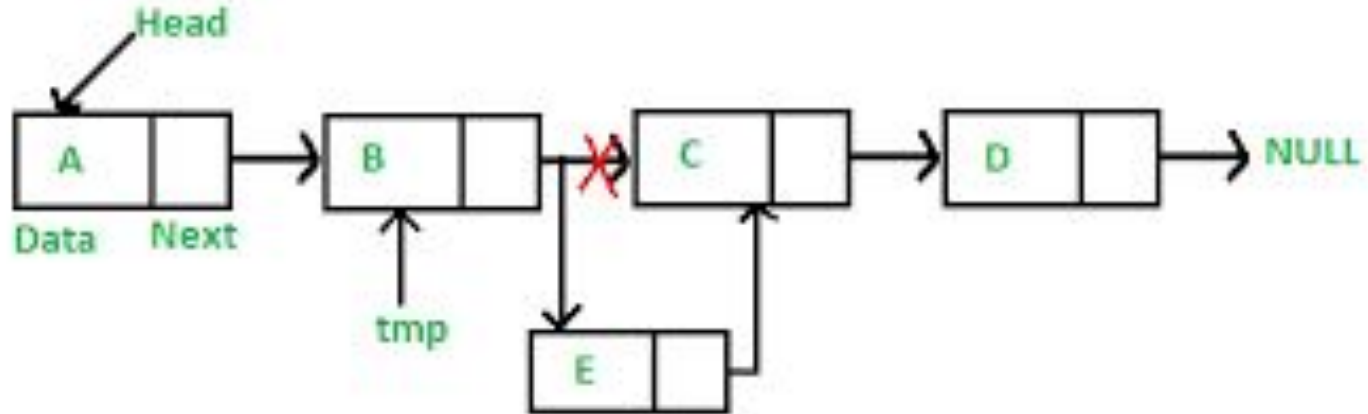
1. Fixed Capacity
2. Insert
3. Removal

Solution? Linked List



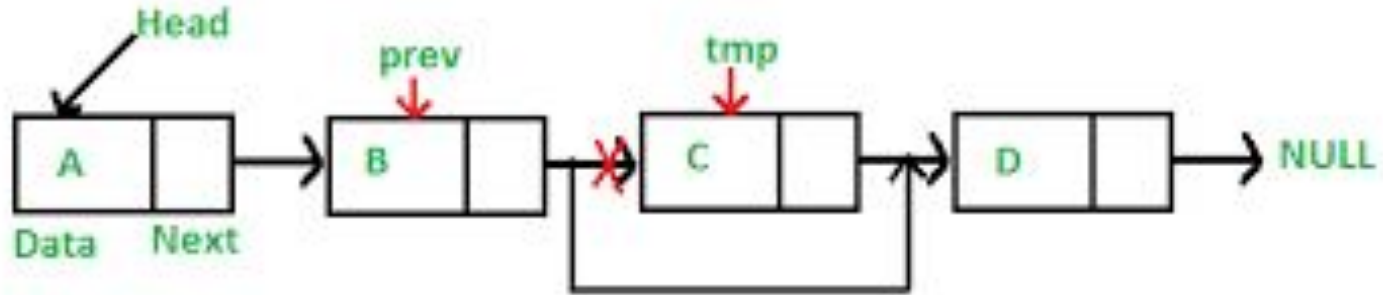
Linked List

1. Easy insert



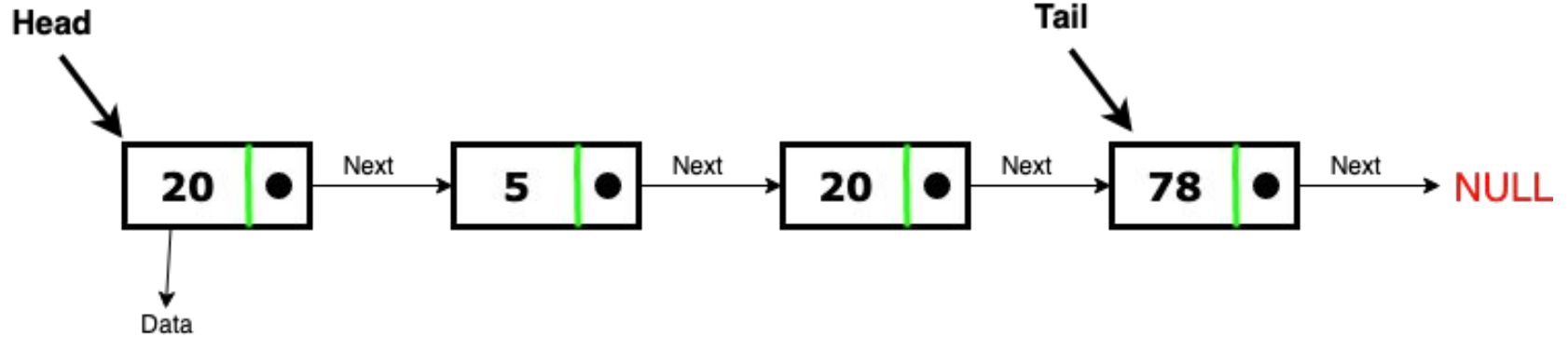
Linked List

2. Easy Removal



Linked List

3. Easy Resize



Linked List - Problems?

1. Random Access
2. Space

Linked List - Initialization

```
1 # Node class design
2 class Node:
3     def __init__(self, e, n):
4         self.elem = e
5         self.next = n
6
```


Linked List - Creation (From an Array)

```
1 # Creating a list
2 def createList(a):
3     head = Node(a[0], None)
4     tail = head
5     for i in range(1, len(a)):
6         n = Node(a[i], None)
7         tail.next = n
8         tail = tail.next
9     return head
```

Linked List - Iteration

```
1 # Iteration over a linked list
2 def iteration(head):
3     temp = head
4     while temp != None:
5         print(temp.element)
6         temp = temp.next
```

Linked List - Count

```
1 # Counting number of element in the list
2 def count(head):
3     count = 0
4     temp = head
5     while temp != None:
6         count += 1
7         temp = temp.next
8     return count
```

Linked List - Get Element

```
1 # Getting element of an specific index
2 def elemAt(head, idx):
3     count = 0
4     temp = head
5     obj = None
6     while temp != None:
7         if count == idx:
8             obj = temp.element
9             break
10        temp = temp.next
11        count += 1
12    if obj == None:
13        print("Invalid index")
14    return obj
```

Linked List - Get Node

```
1 # Getting node of an specific index
2 def nodeAt(head, idx):
3     count = 0
4     temp = head
5     obj = None
6     while temp != None:
7         if count == idx:
8             obj = temp
9             break
10        temp = temp.next
11        count += 1
12    if obj == None:
13        print("Invalid index")
14    return obj
```

Linked List - Update Value at Index

```
1 # Setting new element of an specific index
2 def set(head, idx, elem):
3     count = 0
4     temp = head
5     isUpdated = False
6     while temp != None:
7         if count == idx:
8             temp.elem = elem
9             isUpdated = True
10            break
11        temp = temp.next
12        count += 1
13    if isUpdated:
14        print("Value successfully updated!!!!")
15    else:
16        print("Invalid index")
```